

mini-sidekick — config usage

Centralised settings + logging. Import settings from config.py; never call os.getenv elsewhere.

Module setup (every file starts with this)

```
from config import get_logger
log = get_logger(__name__)
```

Using the settings

```
# db.py
from sqlalchemy import create_engine
from config import DB_PATH, get_logger
log = get_logger(__name__)
engine = create_engine(f"sqlite:///DB_PATH")
log.info("db engine created at %s", DB_PATH)
```

```
# extractor.py
from anthropic import Anthropic
from config import CLAUDE_API_KEY
client = Anthropic(api_key=CLAUDE_API_KEY)
```

```
# charts.py
from config import CHARTS_DIR
out = CHARTS_DIR / f"{name}.png"
fig.savefig(out)
```

Logging levels — pick the right one

```
log.debug("ocr raw text: %r", text)           # verbose, dev-only
log.info("processed receipt: vendor=%s amount=%.2f", v, a) # normal flow
log.warning("low confidence: %.2f", conf)       # noticed, kept going
log.error("ocr returned empty for %s", path)    # broken but contained
log.exception("db write failed")               # inside except: + traceback
```

Node + tool call pattern (every decision traceable)

```
# graph/planner.py
def planner_node(state: dict) -> dict:
    log.info("planner: start image=%s", state["image_path"])
    dup = find_duplicate(state["image_path"])
    log.info("planner: duplicate_check result=%s", bool(dup))
    if dup:
        log.info("planner: routing to skip (dup id=%s)", dup.id)
        return {"**state", "db_result": f"skipped duplicate {dup.id}"}
    log.info("planner: routing to extractor")
    return state

# extractor.py
def extract(ocr_text: str) -> dict:
    log.info("claude call: prompt_chars=%d", len(ocr_text))
    response = client.messages.create(...)
    log.info("claude returned: in_tokens=%d out_tokens=%d",
             response.usage.input_tokens, response.usage.output_tokens)
    return parse(response)
```

Habit to form early — lazy formatting

Always use `log.info("x=%s", x)` over f-strings (`log.info(f"x={x}")`). Interpolation only happens if the level is enabled, so `DEBUG` calls cost nothing in production. Also safer: if `x.__repr__()` raises, the error stays inside logging instead of crashing the actual code path.

Example output line

```
2026-05-25 14:32:01 | INFO      | sidekick.planner | duplicate_check result=False
```